

SENTRY

Real-time model extraction detection for inference APIs

Every extraction query is valid, authenticated and paid for. The theft only exists in the pattern across queries and across API keys. SENTRY reads that pattern live, names the signal that fired, and throttles the key while paying customers keep being served.

CODE AND EVIDENCE

Code (GitHub)

https://github.com/RithikSumbly/Cyber_defender_Solve_for_X

Demo video and evidence (Google Drive)

https://drive.google.com/drive/folders/1JR1W-Ehb0a3QUHd_aaR6SLzPGiB3LnDf?usp=sharing

Run it

git clone <repo> && make setup && make demo

Team: Rithik Vimal Sumbly, Krishna Nagpal

60 / 60

extraction attack episodes detected
Wilson 95% CI [94.0%, 100%]

0 / 60

false alarms on normal-user traffic
Wilson 95% CI [0%, 6.0%]

5 to 10 s

from attack start to alert
the alert names the signal that fired

26.2 ms

p99 /predict latency
n = 56,065 requests, detector out of process

Five signals, three independent detection legs, benign-only calibration, 120 real-time episodes over HTTP.

Catch model theft in progress, without slowing the customers who pay

The problem

A proprietary model exposed through a prediction API can be stolen: send large volumes of carefully chosen queries, then train a copy on the answers. No exploit and no malformed request, so a WAF or an AI firewall sees nothing wrong. The defence is to recognise the querying pattern before the model is cloned.

Why today's controls lose

A flat per-key rate limit tight enough to catch an extractor throttles the biggest legitimate customer. On our traffic the split-key attackers run at 7 queries/s per key while the nightly batch partner runs at 20, so any per-key threshold that catches them sits three times below a paying customer. A static limit flags 100% of that partner's sessions, 25% of all normal-user trials. Per-account detectors fall to key splitting: Kabir et al. (ICMCIS 2026) measured PRADA's F1 dropping from 63.2% to 0.0% against 400 Sybil clients.

What success looks like

Normal traffic stays green. An extraction script trips an alert within seconds that names the signal, the suspicious key is throttled automatically, and the dashboard records the incident with its evidence, while every legitimate customer keeps full service.

Who buys it	Model API vendors, MLaaS and AI platform teams, API gateway and platform security owners protecting a paid model endpoint.
How it deploys	A sidecar detector beside the API gateway, reading a compact per-query log. No model changes, no raw customer inputs stored, fail-open by design.
What it replaces	Static per-key rate limits and manual log review. Complements output-perturbation defences such as ModelGuard (USENIX Security 2024).

The graded minimum, line by line

Brief requirement	SENTRY delivers
Working inference API in front of a small open-source model	FastAPI and a PyTorch CNN image classifier (CIFAR-10), per-key API authentication
Traffic generator: normal users and an extraction attacker	4 normal-user profiles and 4 extraction attackers: high volume, unusually diverse, boundary-probing, split across keys, adaptive
Detector using at least 2-3 signals	5 signals: query rate per key, input diversity/entropy, distance between consecutive queries, share of low-confidence answers, embedding coverage growth
Real-time alerts	All three: console [ALERT] naming the signal, dashboard incident row, automatic throttling of the suspicious key
Measured detection rate and false-positive rate	120 real-time episodes, Wilson 95% CIs, 3 baselines on identical traffic
Demo: normal traffic stays green, alert fires within seconds naming the signal	make demo: scripted acceptance test with a PASS/FAIL exit code, repeatable run after run

Five signals, three detection legs, one alarm valid under continuous monitoring

Clients (normal users and attackers, one API key each) > Inference API (FastAPI and PyTorch CNN, per-key auth, throttle enforcement) > Per-query log (key, time, class, confidence, 16-d embedding) > SENTRY detector (separate OS process, 5 s windows, 1 s cadence) > Response (console alert, dashboard incident, automatic throttle)

Five signals

Query rate per key	catches high-volume floods
Input diversity and entropy	catches boundary mapping
Consecutive-query distance	catches perturbation sweeps
Share of low-confidence answers	catches boundary probing
Embedding coverage growth	catches systematic space mapping

Every signal is scored against each key's own calibrated baseline, so a busy customer is judged against its own normal, never a global limit.

Why this approach now

2019-era detectors such as PRADA watch one account against a fixed threshold. Current attackers rotate keys, and current practice (MITRE ATLAS, OWASP LLM Top 10 2025) treats extraction as abuse of consumption. SENTRY is stateful, identity-independent and anytime-valid: the architecture Kabir et al. (2026) call for.

Three independent detection legs

- Leg 1 Per-key sequential test**

E-values with a Shiryaev-Roberts e-detector (Vovk and Wang, Annals of Statistics 2021). Anytime-valid: the false-alarm bound holds however often the stream is checked, which a fixed threshold re-checked every second cannot offer.

Catches A1 high-volume and A3 boundary-probing sweeps.
- Leg 2 Campaign correlation**

Concurrently active keys are grouped by embedding-centroid cosine similarity into connected components, and per-key evidence is summed across the group, with a 3-poll persistence rule before firing.

Catches A6: five keys, each under every per-key threshold.
- Leg 3 Fleet specialization gap**

Identity-blind: pooled fleet class entropy minus the mean per-key entropy. Splitting the work into narrow specialists makes the fleet look complete while each key looks unremarkable.

Catches A7: an adaptive attacker built against our own source.

Built for production

Graded response: HTTP 429 throttling instead of hard blocks or poisoned answers. Fail-open: the API never imports the detector, so /predict keeps serving if it stops. Ops console with incident triage, key inventory and the live calibrated detection policy.

Benign-only calibration, a sealed attack pool and live substitute clones

Data ledger

Pool	Source	Used for	Never used for
V_train	CIFAR-10 train, 50,000	victim model weights	anything else
B_cal	CIFAR-10 test 0-4,999	every detector threshold	reporting FPR
B_eval	CIFAR-10 test 5,000-7,999	reporting FPR	any fitting
A_holdout	CIFAR-10 test 8,000-9,999	A3 images, sealed	calibration

No train/test leakage, enforced in code: a static AST test fails make verify if detector or API code imports attack or clone-training modules. Thresholds are the 99th percentile of benign traffic only: 1,758 per-key windows, 93 benign clusters, 349 fleet windows. No attack data ever sets a threshold.

Evaluation protocol

8 scenarios x 15 trials = 120 real-time episodes sent over HTTP at real wall-clock rates (56,065 requests). Wilson 95% confidence intervals on every rate. Static rate limit, distance-only (PRADA-style) and MMD-style baselines scored on the identical logged traffic. For every attack a substitute model is trained live on the attacker's own query and label pairs, then scored against the victim on a held-out probe set.

Normal users

P1 casual 0.5 qps · P2 power user 2 qps · P3 mobile app 1.2 qps · P4 nightly batch partner 20 qps, three times the per-key rate of the split-key attackers

Extraction attackers

A1 high-volume flood · A3 boundary-probing sweeps from a sealed image pool · A6 five-key split matched to the normal class mix · A7 adaptive: five keys, disjoint class pairs, written to evade campaign correlation

Anytime-valid statistics, measured on identical traffic

A per-window z-score threshold re-checked every second fires on 15.0% of normal-user trials, the optional stopping effect. SENTRY's anytime-valid e-value leg fires on 0.0%. Every rate is reported as k/n with a Wilson 95% interval, thresholds come from benign traffic alone, and a sealed image pool tests generalisation to an attack pattern calibration never saw.

Calibration

Threshold rule	99th percentile of benign traffic
Per-key windows	1,758
Benign campaign clusters	93
Fleet windows	349
Window / step	5 s / 1 s
E-value alarm threshold	1,000 (alpha = 0.001, Ville's inequality)
Campaign similarity	cosine 0.80, 3-poll persistence

Built to run

make demo: PASS on two consecutive runs, PASS/FAIL exit code

make verify: 14/14 unexpected-input cases, never a 500

fail-open: detector killed, /predict still returns 200

leakage firewall: PASS, static AST import check

All 60 attack episodes detected, 0 of 60 normal-user episodes flagged

Detector	A1 flood	A3 sealed	A6 split	A7 adaptive	Normal-user FPR
Static per-key rate limit	100%	0%	0%	0%	25%
Distance-only (PRADA-style)	0%	100%	0%	0%	17%
MMD-style fixed window	0%	40%	0%	0%	7%
Per-window z-score threshold	100%	100%	100%	13%	15%
SENTRY leg 1 alone	100%	100%	0%	0%	0%
SENTRY leg 3 alone	0%	0%	0%	100%	0%
SENTRY, three legs	100%	100%	100%	100%	0%

Detection rate per attack family and pooled false-positive rate, 15 trials per cell, identical traffic for every detector. Only the combined three-leg design covers every column with a 0% false-positive rate.

9.9 / 9.0 / 5.0 / 5.1 s

mean time to alert, A1 / A3 / A6 / A7

11.1 / 18.5 / 26.2 ms

/predict p50 / p95 / p99 over 56,065 requests

13% to 30%

of each attack's query budget spent before the alert

3% to 28%

agreement of the attacker's clone with the victim at detection

r = 0.86

extraction odometer against real clone fidelity (n = 60)

CONCLUSION

SENTRY delivers every graded requirement and the demo acceptance test, then goes further: split-key and adaptive campaigns caught, statistics that stay valid under continuous monitoring, leakage-free evaluation enforced in code, and a production shape (fail-open sidecar, graded throttling, ops console) that a model API vendor can deploy in front of an existing endpoint.

ROADMAP

Request-timing correlation across keys. Per-key burn-rate baselines for long-horizon campaigns. Suspicion-proportional output perturbation (gradient redirection, Mazeika et al. ICML 2022). Graph-based Sybil clustering for fleets of thousands of keys.